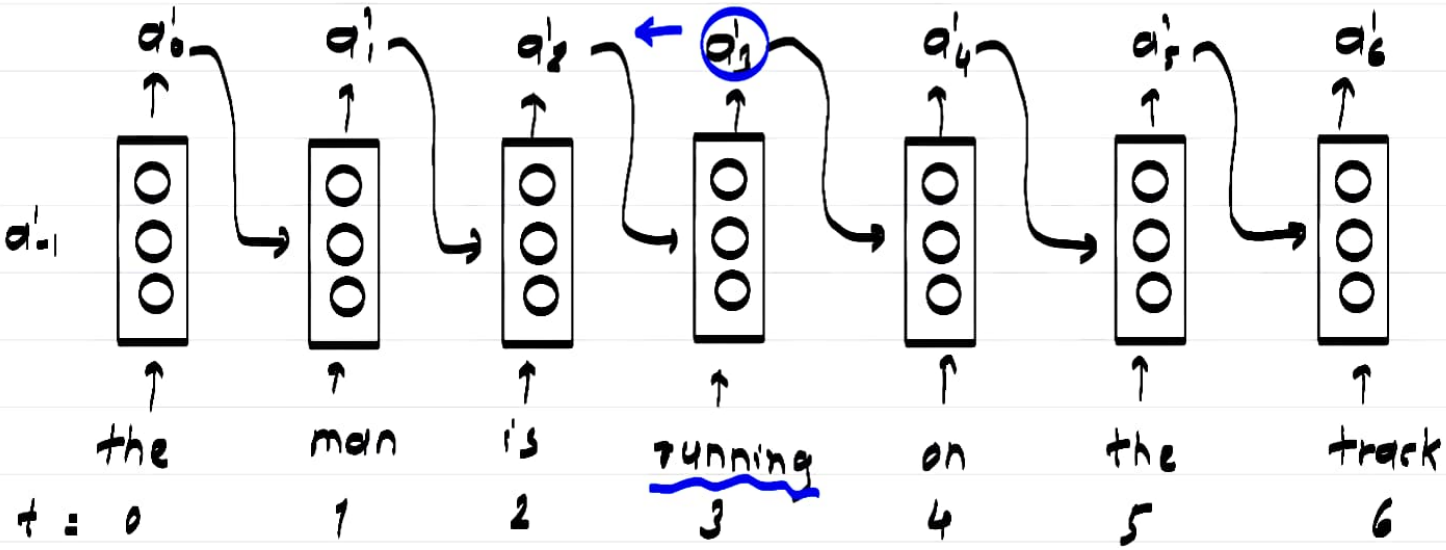
NLP

15-06-26

The man is running on the track.



When tokens are processed in only one direction then its called Unidirectional RNN.

left context

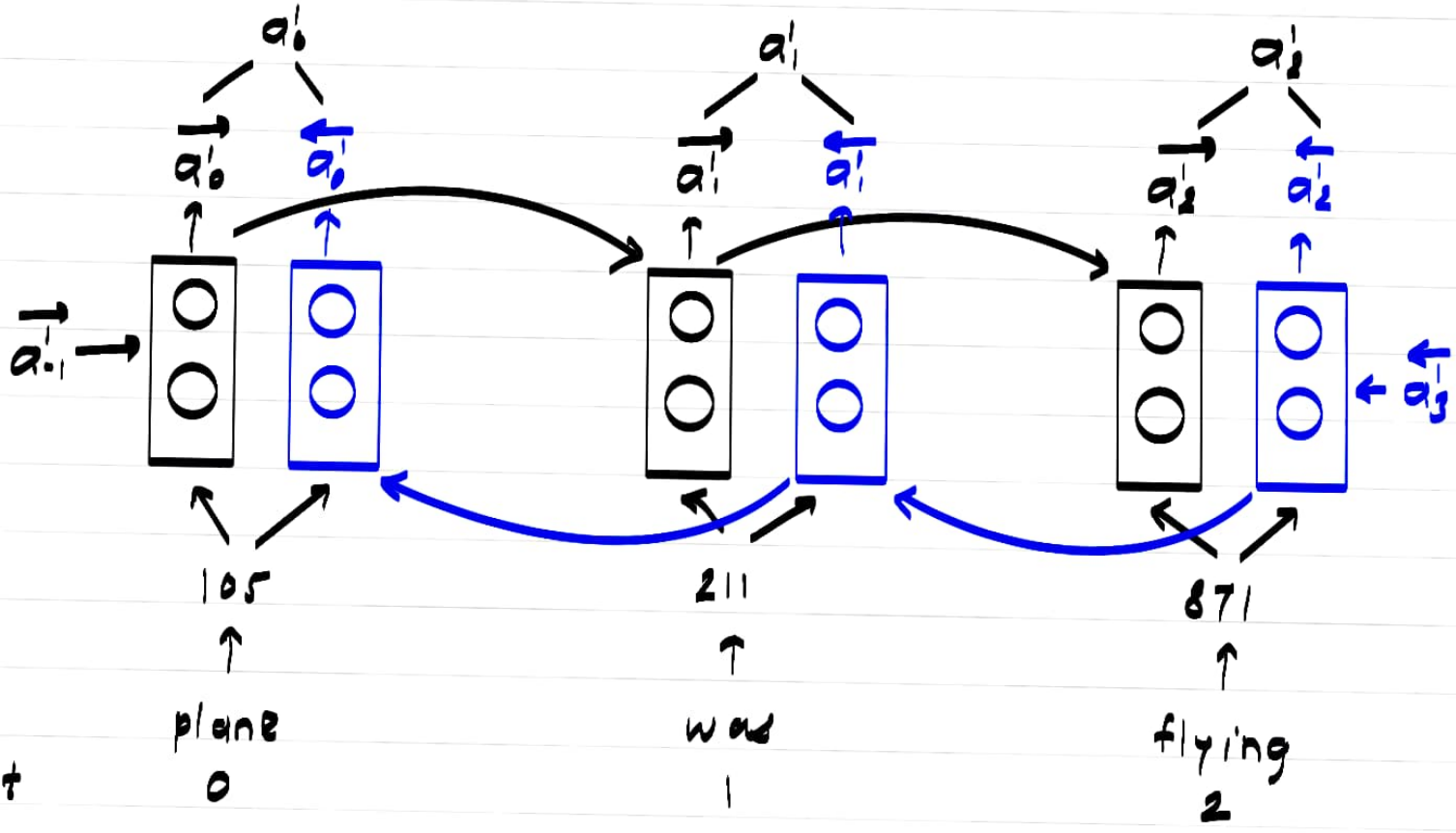
right context

The man is running on the track.

The man is running late to the office.

- meaning of every token depends on other tokens which are present on left as well as right direction.
- left and right both contexts are important.
- We need Bidirectional RNN to capture both contexts

t_1 - Boy is flying a kite. t_3 - Plane was flying.
t_2 - Bird is flying in clouds.



$$\vec{a_0^1} = f(\vec{z_0^1}) = f(\vec{w_x^1}\,105 + \vec{w_a^1}\,\vec{a_{-1}^1} + \vec{b^1})$$

$$\vec{a_1^1} = f(\vec{z_1^1}) = f(\vec{w_x^1}\,211 + \vec{w_a^1}\,\vec{a_0^1} + \vec{b^1})$$

$$\vec{a_2^1} = f(\vec{z_2^1}) = f(\vec{w_x^1}\,871 + \vec{w_a^1}\,\vec{a_1^1} + \vec{b^1})$$

$$\overleftarrow{a_3^1} = f(\overleftarrow{z_3^1}) = f(\overleftarrow{w_x^1}\,871 + \overleftarrow{w_a^1}\,\overleftarrow{a_3^1} + \overleftarrow{b^1})$$

$$\overleftarrow{a_2^1} = f(\overleftarrow{z_2^1}) = f(\overleftarrow{w_x^1}\,211 + \overleftarrow{w_a^1}\,\overleftarrow{a_2^1} + \overleftarrow{b^1})$$

$$\overleftarrow{a_1^1} = f(\overleftarrow{z_1^1}) = f(\overleftarrow{w_x^1}\,105 + \overleftarrow{w_a^1}\,\overleftarrow{a_1^1} + \overleftarrow{b^1})$$

$$a^1_0 = h(z^1_0) = h(\vec{w}^1 \vec{a}^1_0 + \overleftarrow{w}^1 \overleftarrow{a}^1_0 + b)$$

$$a^1_1 = h(z^1_1) = h(\vec{w}^1 \vec{a}^1_1 + \overleftarrow{w}^1 \overleftarrow{a}^1_1 + b)$$

$$a^1_2 = h(z^1_2) = h(\vec{w}^1 \vec{a}^1_2 + \overleftarrow{w}^1 \overleftarrow{a}^1_2 + b)$$

a^1_i → final output for token x_i (was)

$\vec{a}^1_i$ → capturing left context

$\overleftarrow{a}^1_i$ → capturing right context

Bidirectional RNNs give better performance on NLP tasks compared to Unidirectional RNN.

Bidirectional RNNs were able to give very good performance upto texts with near 100 tokens.

As no. of tokens increases RNN performance degrades.

eg. Ramesh (1st token → a^1_0) woke up Sunday Morning to play tennis with seema. He (11th token → a^1_{10}) later went to shopping.

$$a^1_{10} = f(w^1_x x_{10} + w^1_a a^1_9 + b^1)$$

a^1_9 → context of previous 10 tokens

- Due to lot of tokens coming between 'He' and 'Ramesh', the contribution of 'Ramesh' in computation of a_{10} for 'He' will be very less.

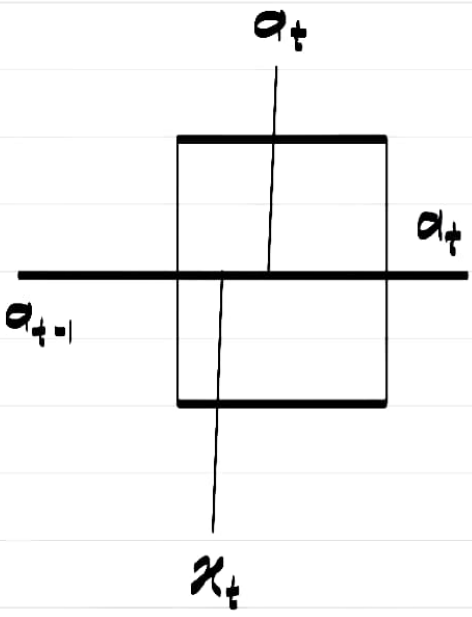
- higher the distance between 2 tokens, lesser will be the context transfer.

- This is very common issue in long sequences.

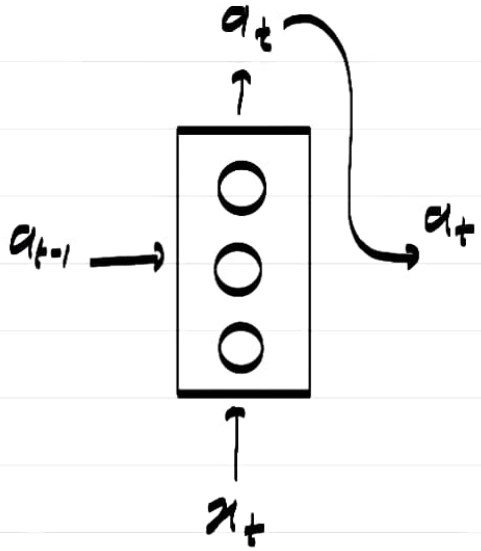
- We need to solve this issue and maintain a strong relationship between distant tokens like 'Ramesh' and 'He'.

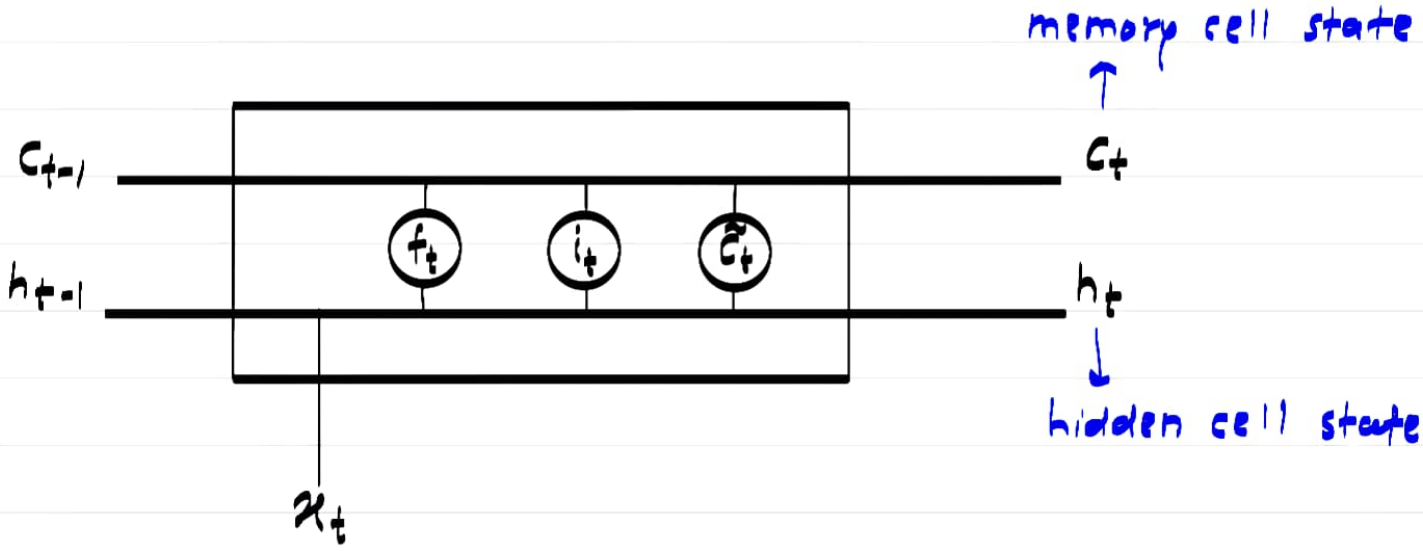
- LSTM architecture solves most of these context issues.

LSTM - Long Short Term Memory RNN :



simple RNN layer





3 types of gates in LSTM:

1) forget gate 2) input gate 3) output gate

1) Forget Gate:

- As sequence length increases, it becomes difficult for RNN to understand.
  1) what part of text to remember
  2) what part of text to forget

in forget gate:

$$f_t = \sigma(z_t) = \sigma\left[\underset{\sim}{W_f}\,(h_{t-1} + x_t) + b_f\right]$$

- σ: sigmoid activation
- W_f: holds context of previous tokens
- x_t: current token

W_f → these weights decide → for given token x_t, which context from h_{t-1} should be remembered and which context should be forgotten

W_f → trainable model parameters
→ during training, to get good model performance model will find optimal values of these weights such that it remembers and forgets the correct contexts.

$$f_t = \sigma(Z_t)$$

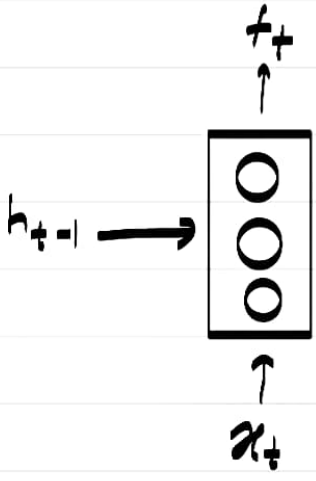
↓

sigmoid $\in [0, 1]$

f_t → n-dimensional vector where all values are between 0-1

where the values are ≈ 0 → gate is trying to forget that context

where the values are ≈ 1 → gate is trying to remember that context.



$$f_t = \sigma(Z_t)$$

$$f_t = \sigma(W_f(h_{t-1} + x_t) + b_f)$$

$$f_t = [f_{t0}, f_{t1}, f_{t2}]$$

what to remember/forget

$$h_{t-1} = [\, h_{t-1}(0) \quad h_{t-1}(1) \quad h_{t-1}(2) \,]$$

$$x_t = [\, x_t \,]$$

$$h_{t-1} + x_t = [\, h_{t-1}(0) \quad h_{t-1}(1) \quad h_{t-1}(2) \quad x_t \,]$$

$$z_t = w_f [\, h_{t-1}(0) \quad h_{t-1}(1) \quad h_{t-1}(2) \quad x_t \,]$$

$$f_t = \sigma(z_t)$$